Support Atelier

ASI

Chapitre 5 : IoC & DI

SOMMAIRE

I. Premier service 2
II. Factorisation du code 2
III. Annexes 3

I. Premier service

1. Au niveau du package **tn.esprit.autoloc**, ajouter une classe d'utilitaires appelée **Utils**.
2. Dans cette classe ajouter une méthode statique nommée **isNotEmpty** qui test si une chaîne de caractères est considérée comme non valide.
3. Créer un nouveau package nommé **tn.esprit.autoloc.exception**.
4. Ajouter dedans la classe d'exception suivante :

```
public class ResourceNotFoundException extends RuntimeException {
    public ResourceNotFoundException(String ressource, String id) {
        super(ressource + " introuvable avec l'identifiant " + id);
    }
}
```

5. Dans le package des services, créer l'interface **IAgenceService** qui contient les méthodes :
   - boolean check(Agence agence, boolean isNew)
   - Agence create(Agence agence)
   - Agence findById(Long id)
   - List<Agence> findAll()
   - Agence update(Long id, Agence agence)
   - void deleteById(Long id)
   - boolean hasVehicules(Long id)
6. Dans le même package, créer une classe d'implémentation nommée **BasicAgenceServiceImpl**, avec des méthodes vides et dans laquelle on utilise :
   - Un attribut de type **IAgenceRepository**
   - L'annotation **Bean** la plus adaptée
   - L'annotation **Lombok** qui permet de créer un constructeur avec les paramètres obligatoires
7. Au niveau de la méthode **create**, tester la valeur de l'identifiant et générer une exception de type **IllegalArgumentException** si cette dernière est remplie. Utiliser la méthode de vérification avant d'insérer l'agence et générer une exception si la vérification échoue.
8. Au niveau de la méthode **findById**, utiliser la méthode **orElseThrow** si l'agence est introuvable.
9. Au niveau de la méthode **update**, utiliser les méthodes **findById** et **chec**k avant de mettre à jour tous les champs.
10. Au niveau de la méthode **deleteById**, générer une exception si l'agence est introuvable (Ne pas récupérer les données).
11. Au niveau de l'interface **AgenceRepositoryMock** ajouter l'annotation Bean la plus appropriée.
12. Au niveau de la classe de tests de l'agence, ajouter un attribut nommé **agenceService** en choisissant le bon type.
13. Ajouter une méthode de test nommée **createAgence**, qui teste la méthode de création du service. Exécuter cette méthode et vérifier le résultat.

II. Factorisation du code

14. Au niveau du package des services, créer une interface générique nommée **IGenericService<T,ID>**. L'interface contient les méthodes déjà implémentées dans la première partie.
15. Ajouter une classe générique abstraite nommé **GenericServiceImpl** qui implémente l'interface **IGenericService** avec le même comportement que l'implémentation de l'agence. En ajoutant les méthodes abstraites suivantes :
    - **getEntityName()** : retourne le nom de l'entité
    - **getId()** : retourne la valeur de l'identifiant d'une entité
    - **setFields** : copie les champs d'une entité vers une autre existante
16. Rendre l'interface **IAgenceService** un cas particulier de **IGenericService**.
17. Créer une deuxième implémentation du service de l'agence nommé **AgenceServiceImpl** en se basant sur **GenericServiceImpl**.
18. Ré-exécuter la méthode de test **createAgence** et vérifier que vous avez une erreur.
19. Résoudre l'erreur et vérifier l'exécution.
20. Compléter les **8** autres services, sachant que le paiement est lecture seule.
21. Pousser le code sur **GitHub**.

III. Annexes

Les annotations de déclaration des beans :

Annotation	Rôle	Couche
`@Component`	Stéréotype générique : la classe devient un bean détecté par le scan.	Transverse
`@Service`	Spécialisation de `@Component` pour la logique métier.	service
`@Repository`	Spécialisation de `@Component` pour l'accès aux données (traduction des exceptions de persistance). Inutile sur une interface Spring Data (Atelier 3).	repository
`@Controller`, `@RestController`	Spécialisation de `@Component` pour la couche web.	web.controller (Atelier 5)
`@Configuration` + `@Bean`	Déclaration **explicite** d'un bean par une méthode : pour une classe qu'on ne peut pas annoter (bibliothèque, classe du JDK) ou une construction particulière.	Configuration
`@Autowired`	Demande l'injection. Facultatif sur un constructeur unique.	Transverse
`@Primary`, `@Qualifier`	Résolution d'ambiguïté quand plusieurs beans ont le même type.	Transverse
`@Scope`	Modifie le scope d'un bean (singleton par défaut).	Transverse

Erreurs courantes du conteneur (message `APPLICATION FAILED TO START`) :

Exception	Cause fréquente	Correction
`NoSuchBeanDefinitionException`	Aucun bean du type demandé : annotation oubliée, classe hors du package de scan	Ajouter `@Service` ou `@Component` ; vérifier le package
`NoUniqueBeanDefinitionException`	Plusieurs beans du type demandé	`@Primary` ou `@Qualifier`
`UnsatisfiedDependencyException`	Enveloppe l'erreur précédente : une dépendance ne peut pas être fournie	Lire la cause la plus profonde (`Caused by`)
Dépendance circulaire (`BeanCurrentlyInCreationException`)	Deux beans qui s'injectent mutuellement	Revoir la conception : extraire la responsabilité commune